

|                                                                                   |            |                                 |                    |
|-----------------------------------------------------------------------------------|------------|---------------------------------|--------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 1 de 13     |
|                                                                                   | Versió: 01 | DAM0489_EAC3_Enunciat_2526S2    | Lliurament 11-5-26 |

**Desenvolupament d'aplicacions multiplataforma**  
**Mòdul 0489 – Programació multimèdia i dispositius mòbils**  
**Unitat 3**  
**EAC3**  
**(Curs 2025/26 2n semestre)**

**Enunciat**

**1) 2 Punts**

Contesta el qüestionari associat a l'EAC3 en el Moodle (***Qüestionari de l'EAC3***).

Inclou al fitxer comprimit una captura de pantalla del qüestionari acabat.

**Resum dels vostres intents anteriors**

**Intent 1**

**Estat** Acabat  
**Començat el** dijous, 23 d'abril 2026, 17:58  
**Completat el** dilluns, 11 de maig 2026, 23:11  
**Durada** 18 dies 5 hores

11/05/2026 23:55 disponible

**ATENCIÓ:**

- El qüestionari només té 1 intent. No té, però, límit de temps.
- Cada resposta correcta suma 1/n punts on «n» és el nombre de preguntes, cada error en resta 1/2n punts.

|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 2 de 13          |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |

## 2) 8 Punts

### Apartat 1: Moviment bàsic del submarinista i càmera (15%)

#### 1. Moviment del personatge

Implementa el moviment bàsic del submarinista dins del mapa.

El moviment ha de simular flotació sota l'aigua:

quan el jugador impulsa el personatge, aquest puja,

quan no l'impulsa, baixa lentament.

El personatge també s'ha de poder desplaçar lateralment pel mapa.

El mapa ha de ser més gran que la pantalla del dispositiu.

#### Consells

Utilitza **TiledMap** i **TiledMapRenderer** per gestionar el mapa.

Pots controlar la pujada i la baixada modificant la velocitat vertical del personatge.

#### Apartat Fet?

Cuando se presiona un lado de la pantalla el jugador ira a esa dirección pero a la vez subirá.

```
Lógica de movimiento en Player.kt (método update)
fun update(delta: Float, isTouched: Boolean, isUpPressed: Boolean) {
    stateTime += delta

    Control vertical (impulso vs gravedad)
    if (isTouched || isUpPressed) {
        velocity.y += 700f * delta
        if (velocity.y > 250f) velocity.y = 250f
        state = State.UP
    } else {
        velocity.y -= gravity * delta
        if (velocity.y < -200f) velocity.y = -200f
        state = if (velocity.y < -20f) State.DOWN else State.IDLE
    }

    Aplicar movimiento y fricción horizontal
    position.x += velocity.x * delta
    position.y += velocity.y * delta
    velocity.x *= 0.9f
}
= if (velocity.y < -20f) State.DOWN else State.IDLE
}
```

|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 3 de 13          |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |

## 2. Càmera que segueix el personatge

- Fes que la càmera segueixi el submarinista mentre es desplaça pel mapa.
- El personatge s'ha de mantenir dins de la vista de la càmera.
- Ajusta els límits de la càmera perquè no mostri zones fora del mapa.

### Consells

- Utilitza `OrthographicCamera` per gestionar el seguiment del personatge.

### Apartat Fet?

La càmera es de tipo `OrthographicCamera`, que es la que se usa para juegos 2D. En `GameScreen.kt` la configuro para que siempre apunte a donde está el jugador, y con `MathUtils.clamp` me aseguro de que nunca salga fuera de los límites del mapa.

```
Seguimiento y límites de cámara en GameScreen.kt (método render)
camera.position.set(player.position.x, player.position.y, 0f)

Cálculo de límites según el tamaño del mapa cargado
val mapaAnchoPíxeles = mapLayer.width * mapLayer.tileWidth
val mapaAltoPíxeles = mapLayer.height * mapLayer.tileHeight

Restricción de la cámara para no mostrar el vacío exterior
camera.position.x = MathUtils.clamp(camera.position.x, camera.viewportWidth / 2f, mapaAnchoPíxeles - camera.viewportWidth / 2f)
camera.position.y = MathUtils.clamp(camera.position.y, camera.viewportHeight / 2f, mapaAltoPíxeles - camera.viewportHeight / 2f)
camera.update()
```

|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 4 de 13          |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |

### 3. Parets i col·lisions amb l'entorn

- Afegeix parets, roques, sostre, terra o altres límits al mapa.
- El personatge no ha de poder travessar aquests elements.
- Cal implementar col·lisions amb l'entorn.

#### Consells

- Pots utilitzar `Rectangle` i comprovar interseccions per detectar col·lisions.

#### Apartat Fet?

Las colisiones con el mapa las detecto leyendo las capas del TiledMap ("Paredes", "Paredes2", "Obstaculos"). Compruebo si el rectángulo del jugador se solapa con algún tile sólido, y si es así, cancelo el movimiento.

```
Detección de colisiones por Tiles en GameScreen.kt
private fun comprobarColisionesGenerico(p: Player): Boolean {
    val playerRect = p.getBoundingBox()
    val startX = (p.position.x / 32).toInt()
    val startY = (p.position.y / 32).toInt()

    Comprobamos los tiles en una rejilla de 2x2 alrededor del jugador
    for (x in startX..startX + 1) {
        for (y in startY..startY + 1) {
            val c1 = mapLayer.getCell(x, y)
            val c2 = mapLayer2?.getCell(x, y)
            if ((c1 != null && c1.tile != null) || (c2 != null && c2.tile !=
null)) {
                rectBloque.set(x * 32f, y * 32f, 32f, 32f)
                if (playerRect.overlaps(rectBloque)) return true
            }
        }
    }
    return false
}
```

## Apartat 2: Animacions del personatge (15%)

### 1. Animacions del submarinista

- Implementa animacions del personatge segons el seu estat de moviment.
- Com a mínim, cal mostrar animacions diferenciades per a:
  - pujada,
  - descens,
  - desplaçament lateral o repòs.
- Les animacions poden ser seqüències de sprites.
- Pots utilitzar sprites open source, no cal fer-los manualment.

|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 5 de 13          |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |

## Consells

- **Utilitza** `Animation<TextureRegion>` **per gestionar les animacions.**
- **Pots trobar recursos gratuïts a llocs com** `opengameart.org` **o** `itch.io`.
- <https://opengameart.org/content/scuba-diver>
- <https://ansimuz.itch.io/underwater-diving>

## Apartat Fet?

He implementado dos animaciones para el jugador usando `Animation<TextureRegion>` de libGDX:

- `animIdle` — cuando está quieto o bajando (usa `player-idle.png`)
- `animSwimming` — cuando sube o se mueve en horizontal (usa `player-swimming.png`)

```
Gestión de frames y dibujo en GameScreen.kt
val currentAnim = if (player.state == Player.State.UP || player.velocity.x != 0f) animSwimming else animIdle
val currentFrame = currentAnim.getKeyFrame(player.stateTime)

game.batch.begin()
Invertir el sprite según la dirección horizontal
if ((player.velocity.x < 0) != currentFrame.isFlipX) {
    currentFrame.flip(true, false)
}
game.batch.draw(currentFrame, player.position.x, player.position.y,
player.size, player.size)
game.batch.end()
```

|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 6 de 13          |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |

### Apartat 3: Recollida d'objectes (15%)

#### 1. Tipus d'objectes

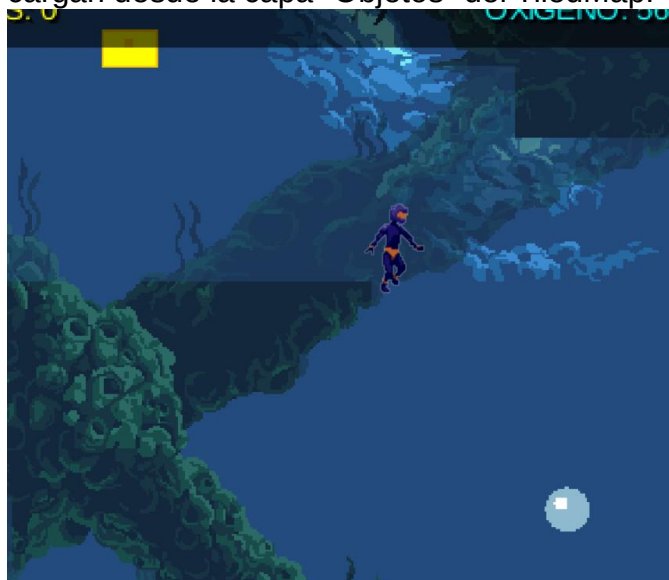
- Afegeix dos tipus d'objectes dispersos pel mapa que el jugador pugui recollir:
  - Tresors: cada vegada que el jugador en recull un, augmenten els punts.
  - Bombolles d'oxigen: quan es recullen, augmenta l'oxigen del jugador.

#### Consells

- Pots fer servir `Array<Sprite>` o una estructura equivalent per emmagatzemar els objectes.
- Utilitza col·lisions amb `Rectangle` per determinar quan el personatge recull un objecte.

#### Apartat Fet?

He creado dos tipos de objetos recogibles definidos en la clase `Item.kt` con un enum `ItemType`: `TREASURE` (da puntos) y `OXYGEN` (recupera aire). Los objetos se cargan desde la capa "Objetos" del `TiledMap`.



|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 7 de 13          |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |

## 2. Implementació de col·lisions amb objectes

- Implementa la col·lisió entre el submarinista i els objectes del mapa.
- Quan el personatge toca un objecte:
  - aquest s'ha de recollir i desaparèixer,
  - s'ha de modificar la puntuació o l'oxigen segons el tipus d'objecte.

### Consells

- Utilitza `Intersector.overlaps()` per comprovar les col·lisions.
- Pots utilitzar `removeValue()` o una solució equivalent per eliminar objectes recollits.

### Apartat Fet?

Para detectar si el jugador recoge un objeto uso `Intersector.overlaps()`, que comprueba si dos rectángulos se tocan:

```
Gestió de la lista de objetos con Iterator en GameScreen.kt
val itemIter = items.iterator()
while (itemIter.hasNext()) {
    val item = itemIter.next()
    game.batch.draw(item.region, item.position.x, item.position.y, item.size,
item.size)

    Comprobar solapamiento usando Intersector
    if (com.badlogic.gdx.math.Intersector.overlaps(player.getBoundingRectan-
gle(), item.getBoundingRectangle())) {
        if (item.type == ItemType.TREASURE) {
            player.score += 1
            player.treasuresCollected++
        } else {
            player.oxygen = Math.min(100f, player.oxygen + 20f)
        }
        itemIter.remove()
    }
}
```

## Apartat 4: Mines submarines i detecció de proximitat (15%)

### 1. Mines en moviment

- Afegix mines submarines dins del mapa.
- Les mines s'han de moure de manera repetitiva amunt i avall entre dos punts.
- El moviment ha de ser automàtic.

### Consells

- Utilitza `Vector2` o variables equivalents per gestionar el moviment vertical de les mines.

|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 8 de 13          |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |



### Apartat Fet?

```
fun update(delta: Float) {
    stateTime += delta
    if (state == MinaState.ACTIVE) {
        Movimiento automático arriba y abajo
        position.y = startPosition.y + MathUtils.sin(stateTime * speed + off-
set) * range
    } else if (state == MinaState.EXPLODING) {
        if (explosionAnim.isAnimationFinished(stateTime - explosionStartTime))
    {
        state = MinaState.REMOVED
    }
    }
}
```

## 2. Activació per proximitat

- Implementa un sistema perquè la mina detecti si el jugador s'hi acosta massa.
- Quan el submarinista entra dins d'una certa distància, la mina s'ha d'activar o explotar.

### Consells

- Pots calcular la distància entre la posició del jugador i la de la mina.
- Defineix un radi de proximitat a partir del qual la mina reacciona.

### Apartat Fet?

Cuando el jugador se acerca a menos de 60 píxeles del centro de la mina, esta explota y le quita 30% de oxígeno. Luego se reproduce la animación de explosión y cuando termina, la mina desaparece.

```
Detección de proximidad en GameScreen.kt
val dist = Vector2.dst(player.position.x + player.size/2, player.position.y +
player.size/2,
                        mina.position.x + mina.size/2, mina.position.y +
mina.size/2)

if (mina.state == MinaState.ACTIVE && dist < 60f) {
    mina.state = MinaState.EXPLODING
    mina.explosionStartTime = mina.stateTime
    player.oxygen -= 30f
}
```

## 3. Explosió i radi de dany

- Quan una mina explota, ha de fer dany si el submarinista es troba dins del radi de l'explosió.

|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 9 de 13          |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |

- L'explosió ha de tenir efecte sobre el jugador encara que no hi hagi contacte directe amb la mina.
- Pots fer que la mina desaparegui després d'explotar, o bé que deixi de ser activa.

#### Consells

- Pots representar el radi d'explosió amb Circle o una estructura equivalent.

#### Apartat Fet?

```
Representación visual de la explosión y daño radial
if (mina.state == MinaState.EXPLODING) {
    val explosionFrame = mina.explosionAnim.getKeyFrame(mina.stateTime -
mina.explosionStartTime)
    game.batch.draw(explosionFrame, mina.position.x - 16f, mina.position.y -
16f, mina.size + 32f, mina.size + 32f)
}
```

## Apartat 5: Oxigen, dany i interfície (15%)

### 1. Pèrdua d'oxigen

- El submarinista ha de perdre oxigen de manera progressiva amb el pas del temps.
- També ha de perdre oxigen:
  - quan rep dany per una explosió,
  - i quan col·lideix amb elements de l'entorn, si així ho has implementat.
- Aquest sistema ha de forçar el jugador a moure's i a recollir bombolles d'oxigen.

#### Consells

- Pots utilitzar una variable numèrica per representar l'oxigen actual del jugador.
- Pots limitar el valor màxim d'oxigen.

#### Apartat Fet?

El oxígeno se gestiona en Player.kt. Se va gastando poco a poco cada segundo (2% por segundo), se pierde cuando la mina explota cerca (30%), y se recupera recogiendo burbujas (+20%).

```
Desgaste de oxígeno en Player.kt
oxygen -= 2f * delta
if (oxygen < 0f) oxygen = 0f
```

|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 10 de 13         |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |

## 2. Interfície de joc

- Mostra a la pantalla la informació bàsica de la partida.
- Com a mínim, cal mostrar:
  - l'oxigen actual,
  - la puntuació del jugador.

### Consells

- Utilitza `Label` o eines equivalents de `Scene2D` per mostrar aquesta informació.

### Apartat Fet?

El HUD lo dibujo con una cámara separada (`hudCamera`) para que no se mueva con el mundo. Muestro los puntos y el oxígeno, y cuando el oxígeno baja del 30% el texto cambia de color cian a rojo para avisar al jugador.

```
Dibujado del HUD con cámara fija en GameScreen.kt
private fun drawHUD() {
    game.batch.projectionMatrix = hudCamera.combined
    game.batch.begin()
    drawTextWithShadowHUD("PUNTOS: ${player.score}", 32f, hudViewport.world-
Height - 15f, colorTextoHUD)

    val oxyColor = if (player.oxygen > 30) colorOxigenoOk else colorOxigeno-
Critico
    drawTextWithShadowHUD("OXÍGENO: ${player.oxygen.toInt()}%", hudView-
port.worldWidth - 250f, hudViewport.worldHeight - 15f, oxyColor)
    game.batch.end()
}
```

## Apartat 6: Final de partida (15%)

### 1. Derrota quan s'acaba l'oxigen

- El joc ha d'acabar quan l'oxigen arriba a zero.
- El jugador ha de perdre la partida quan ja no li quedi oxigen.

### Apartat Fet?

Cuando el oxígeno llega a 0, el juego cambia automáticamente a `GameOverScreen`. Esta pantalla muestra fondo rojo oscuro, el título "GAME OVER" y las estadísticas de la partida. Después de 3 segundos aparece la opción de reintentar para evitar que el jugador cambie de pantalla por accidente.

```
Cambio de pantalla por muerte en GameScreen.kt
if (player.oxygen <= 0) {
    game.screen = GameOverScreen(game, player.score, player.timeSurvived,
player.treasuresCollected)
    return
}
```

|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 11 de 13         |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |

}

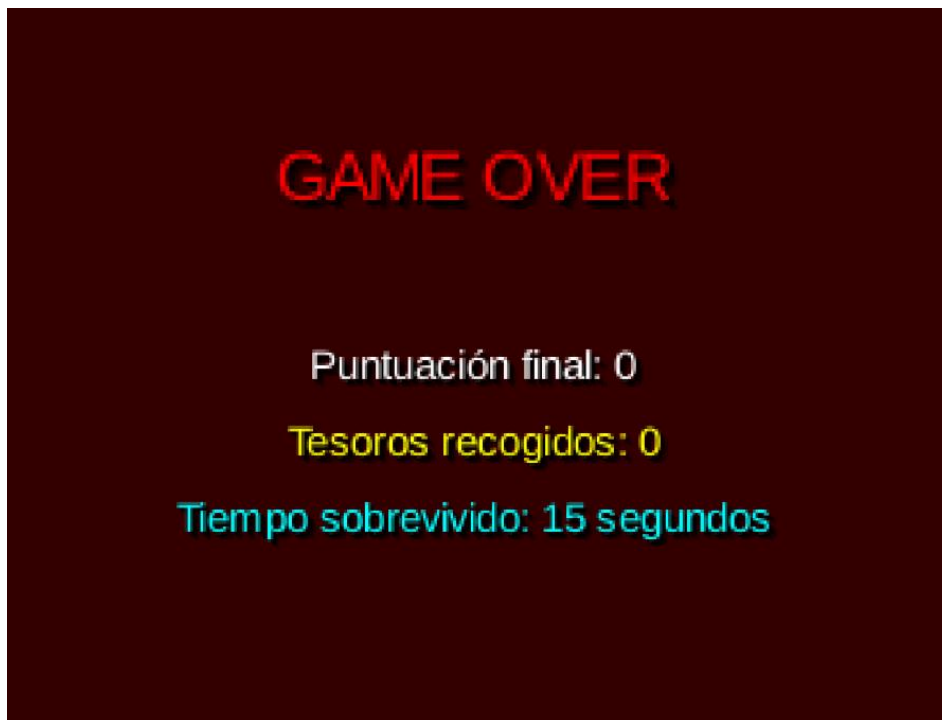
## 2. Pantalla o missatge de final de partida

- Mostra un missatge o pantalla de final de partida que indiqui clarament que el jugador ha perdut perquè s'ha quedat sense oxigen.

### Consells

- Pots utilitzar Scene2D per gestionar la interfície del missatge de final de partida.

### Apartat Fet?



## Apartat 7: Pantalla de resultats del joc (10%)

### 1. Resultats finals

- Afegeix una pantalla de resultats que mostri, com a mínim:
  - la puntuació final del jugador,
  - el temps total que ha sobreviscut,
  - el nombre de tresors recollits.

Això ha de permetre donar context al rendiment del jugador durant la partida.

### Consells

- Utilitza Screen per crear una pantalla de resultats.
- Preferences pot ser una bona manera de guardar la millor puntuació per a partides futures.

|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 12 de 13         |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |

**Apartat Fet?**

Cuando las monedas llegan al total, el juego cambia automáticamente a VictoryScreen.

Lógica de victoria por recogida total en GameScreen.kt

```
if (totalMonedas > 0 && player.treasuresCollected >= totalMonedas) {
    game.screen = VictoryScreen(game, player.score, player.timeSurvived,
    player.treasuresCollected)
    return
}
```

VictoryScreen.kt:

```
drawTextWithShadow(";Has recogido todos los tesoros!", 400f, colorMensaje)
drawTextWithShadow("Puntuación final: $score", 320f, colorPuntos)
drawTextWithShadow("Tiempo sobrevivido: ${time.toInt()} segundos", 220f, co-
lorTiempo)
```

**Entrega**

Cal entregar dues coses:

**1. Un document PDF**

Aquest PDF ha de contenir:

- aquest mateix enunciat,
- una breu explicació de què has fet a cada apartat i subapartat,
- captures de pantalla de l'aplicació en funcionament,
- fragments de codi relacionats amb cada requisit,
- la indicació clara de quin punt s'està mostrant,
- i una marca indicant si cada apartat funciona o no.

**2. El projecte complet**

Cal entregar també el projecte complet de libGDX i Kotlin, sense la carpeta build.

|                                                                                     |            |                                 |                         |
|-------------------------------------------------------------------------------------|------------|---------------------------------|-------------------------|
|  | Codi: I71  | Exercici d'avaluació contínua 3 | Pàgina 13 de 13         |
|                                                                                     | Versió: 01 | ASX0372_EAC3_Enunciat_2526S1    | Lliurament<br>11/5/2026 |